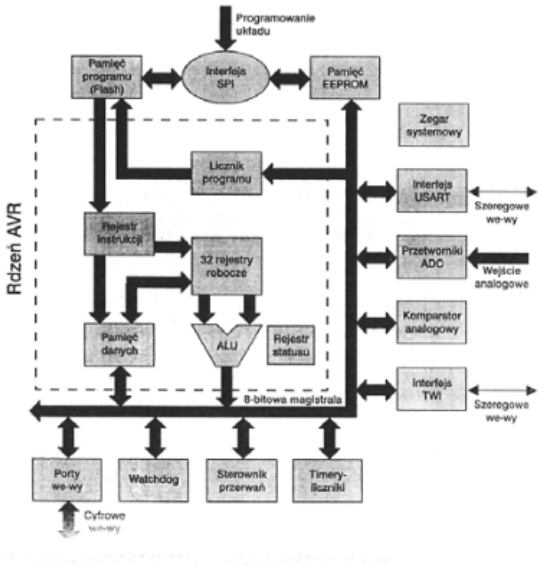
Jednostka centralna – procesor (CPU, rdzeń)



Schemat blokowy procesora rdzeniowego ATmega16

Głównym zadaniem JC jest zapewnienie poprawnego i szybkiego wykonywania programu.

Zadania JC:

- dostęp do pamięci,
- wykonywanie operacji arytmetycznych, logicznych, itp. (**ALU**)
- sterowanie układami peryferyjnymi,
- obsługa przerwań.

Działanie JC polega na cyklicznym wykonaniu **instrukcji** zawartych **w programie** użytkownika przechowywanym w **pamięci**.
Lista instrukcji jest z góry określona dla danego mk.
Pojedyncza **instrukcja** składa się z **kodu operacji** (kodu rozkazu**)** oraz jednego lub kilku **argumentów.**

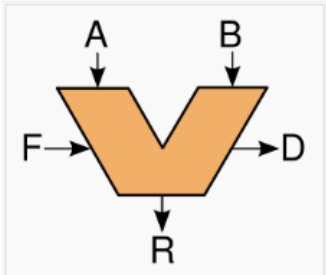
Cykl wykonania instrukcji:

- wczytanie do **wewnętrznych rejestrów JC** kolejnego rozkazu,
- pobranie z pamięci i umieszczenie w odpowiednich **wewnętrznych rejestrach JC** niezbędnych argumentów rozkazu (o ile są potrzebne),
- wykonanie instrukcji – **ALU** + **układ sterowania**

Jednostka arytmetyczno-logiczna ALU

Operacje jednostki ALU:

- działania arytmetyczne
  - dodawanie +
  - odejmowanie –
  - porównywanie liczb
  - zmiana znaku
- działania logiczne
  - suma (OR)
  - iloczyn (AND)
  - suma modulo 2 (XOR)
  - negacja
- przesunięcia w lewo i prawo
- działania na bitach
  - ustawienie bitu
  - zerowanie bitu
  - testowanie bitu
  - negacja bitu



Typowy symbol ALU: A i B - operandy; R - wyjście; F - wejście z jednostki kontrolnej; D - status wyjścia

Tryby adresowania:

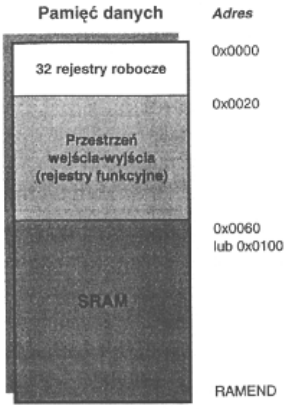
- **Adresowanie natychmiastowe** – operand (argument) jest podany w jawnej postaci w kodzie instrukcji. Tryb ten może być używany wyłącznie w odniesieniu do stałych zapisanych w kodzie programu.
- **Adresowanie bezpośrednie** – adres argumentu (umieszczonego w pamięci danych) znajduje się bezpośrednio po kodzie rozkazu.
- **Adresowanie indeksowe** polega na obliczeniu adresu przez sumowanie zawartości specjalnie przeznaczonego do indeksowania rejestru, nazywanego **rejestrem indeksowym**, z adresem bezpośrednim, zapisanym w instrukcji. Obliczony w ten sposób adres fizyczny pamięci nazywany jest **adresem efektywnym**. Adresowanie to jest szczególnie użyteczne przy operowaniu na blokach danych. Umieszczając w instrukcji adres początku bloku danych można uzyskać wygodny dostęp do kolejnych bajtów danych przez tylko samą zmianę zawartości rejestru indeksowego.
- **Adresowanie pośrednie** – część adresowa instrukcji wskazuje na komórkę pamięci zawierającą adres efektywny.
- **Adresowanie zawartością rejestrów** (odmiana adresacji pośredniej) – adres efektywny jest zawarty w przeznaczonym do tego celu rejestrze lub parze rejestrów; identyfikacja tych rejestrów odbywa się na podstawie słowa rozkazowego.
- **Adresowanie względne** – adres efektywny uzyskiwany jest jako suma części adresowej rozkazu i adresu aktualnie wykonywanej instrukcji zapisanego w tzw. liczniku programu (rozkazów); wykorzystywane w instrukcjach skoków.

Architektury procesorów

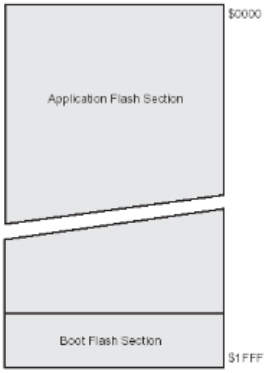
Architektury procesorów rdzeniowych można sklasyfikować według:

- typu **mapy pamięci**
- typu **listy instrukcji**.

Mapa pamięci w sposób graficzny przedstawia rozmieszczenie poszczególnych pamięci w **przestrzeni adresowej** JC. Oprócz adresów obszarów RAM, ROM i innych rodzajów pamięci, mapa ta podaje usytuowanie rejestrów uniwersalnych, adresów procedur obsługi przerwań, rejestrów układów we/wy.



Przestrzeń adresowa pamięci danych



Przestrzeń adresowa pamięci programu

W zależności od **typu struktury** mapy pamięci, procesory rdzeniowe mogą mieć następującą architekturę:

- architekturę harwardzką,
- architekturę Von-Neumanna.

Architektura harwardzka:

- **dwie oddzielne magistrale (szyny) dla danych i rozkazów**, dzięki czemu w trakcie pobierania argumentów wykonywanej właśnie instrukcji można równocześnie zacząć pobieranie następnego słowa rozkazowego. Skraca to cykl rozkazowy i zwiększa szybkość pracy.
- **Obszary adresowe** pamięci danych i programu są **rozdzielone.** W efekcie mamy niejednoznaczność adresów - JC pod tym samym adresem widzi pamięć RAM i ROM - stosuje się inne rozkazy dla pamięci programu i inne dla pamięci danych.
- magistrala danych i rozkazów mają **różną szerokość (długość słowa).**

Wada - utrudniony przepływ danych z pamięci programu do obszaru pamięci operacyjnej - nie jest możliwe indeksowane przesłanie danych z pamięci ROM do RAM, co oznacza np. brak możliwości budowy tabel współczynników stałych w pamięci ROM.

Modyfikacje architektury harwardzkiej:

- **szerokość magistrali programu jest krotnością szerokości magistrali danych**, (dla AVR magistrala danych jest 8-bitowa, a programu 16-bitowa (2 * 8 bitów).
- wprowadza się dodatkowe **bloki pośredniczące** w wymianie danych między pamięcią programu a pamięcią danych.
- rozszerza się listę instrukcji o **dodatkowe polecenia** obsługujące transfer danych z/do pamięci programu, np. rozkaz Load Program Memory LPM.